

Workshop on PYTHON Programming

Dr. Venkataramana Veeramsetty
Assistant Professor
Department of Electrical and Electronics Engineering

S R Engineering College Warangal

June 19, 2019





- PYTHON is a general purpose, object oriented and high level programming language
- It was developed by Guido Van Rossum during 1985-1990
- Python is named after a TV Show called 'Monty Python's Flying Circus'
- PYTHON3 was developed in 2008



✓ Python is Interpreted

- Python is processed at runtime by the interpreter. You do not need to compile your program before executing it.

✓ Python is Interactive

- You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

✓ Python is Object-Oriented

- Python supports Object-Oriented style or technique of programming that encapsulates code within objects.

✓ Python is a Beginner's Language

- Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.



- PYTHON 1.0 released in 1994
- PYTHON 2.0 released in 2000
- PYTHON 2.7.11 is the latest edition of Python 2
- PYTHON 3.0 released in 2008
- PYTHON 3.5.1 is the latest edition of Python 3



CODE-1

```
print(" Hi" )  
print(" Venkat" )
```

OUTPUT-1

Hi
Venkat

CODE-2

```
print(" Hi",end=" " )  
print(" Venkat" )
```

OUTPUT-1

Hi Venkat



Use 'input' command to read input from keyboard
The value which is given through keyboard will be treated as string

CODE

```
x=input('enter value of x')  
print(x)
```

OUTPUT

```
enter value of x10  
10
```

CODE

```
y=input('enter value of y')  
print("the value of y =",y)
```

OUTPUT

```
enter value of y10  
the value of y= 10
```



- In PYTHON2, **input**(or) *rawinput* command used to read a value to the variable through keyboard
- If the value given from the keyboard with " or "" using **input** command then the variable takes that value as a string otherwise it will consider as number
- If the value given from the keyboard with " or "" using *rawinput* command then the variable takes that value as a string always
- Whereas in PYTHON3, only **input** command used, it considers the input value as a string always



CODE

```
x=input('enter value of x')
print(" the value of x=",x)
y=input('enter value of y')
print(" the value of y=",y)
z=input('enter value of z')
print(" the value of z=",z)
a=x+y+z
print(" the value of a=",a)
```

OUTPUT

```
enter value of x10
the value of x= 10
enter value of y20
the value of y= 20
enter value of z30
the value of z= 30
the value of a= 102030
```



- In Python 2, the result of division of two integers is rounded to the nearest integer.
- As a result, $5/2$ will show 2.
- In order to obtain a floating-point division, numerator or denominator must be explicitly used as float.
- Hence, either $5.0/2$ or $5/2.0$ or $5.0/2.0$ will result in 2.5

Python 3 evaluates $5 / 2$ as 2.5 by default, which is more intuitive for new programmers.



- ✓ Python does not use braces({}) to indicate blocks of code for class and function definitions or flow control.
- ✓ Blocks of code are denoted by line indentation, which is rigidly enforced.
- ✓ The number of spaces in the indentation is variable, but all statements within the block must be indented the same amount

```
Created on Wed Mar  6 10:05:11 2019
```

```
@author: SWETHAVENKAT  
"""
```

```
if True:  
    print ("True")
```

```
else:  
    print ("False")
```



Quotation in Python

- ✓ Python accepts single ('), double (") and triple (''' or """) quotes to denote string literals, as long as the same type of quote starts and ends the string.
- ✓ The triple quotes are used to span the string across multiple lines

```
1 # -*- coding: utf-8 -*-  
2 ""  
3 Created on Wed Mar 6 10:05:11 2019  
4  
5 @author: SWETHAVENKAT  
6 ""  
7 singlequote='word'  
8 print(singlequote)  
9 doublequote="my name is venkat"  
10 print(doublequote)  
11 triplequote='''my name is venkat  
12 roll no 02341'''  
13 print(triplequote)
```



- ✓ # is used at stating of the comment line
- ✓ Use triple quotes if there is more comment lines

```
1 # -*- coding: utf-8 -*-  
2 """  
3 Created on Wed Mar  6 10:05:11 2019  
4  
5 @author: SWETHAVENKAT  
6 """  
7  
8 print('Hi Vankat') #Say Hi to venkat  
9 print('roll number = 259') # Roll no of the candidate  
10  
11 '''  
12 Comments in  
13 multiple lines  
14 '''
```

Multiple Statements on a Single Line

```
17 string1='Venkat';string2='Ramana';string3=string1+string2  
18 print(string3)
```



- Variables are nothing but reserved memory locations to store values. It means that when you create a variable, you reserve some space in the memory.
- Based on the data type of a variable, the interpreter allocates memory and decides what can be stored in the reserved memory.
- Therefore, by assigning different data types to the variables, you can store integers, decimals or characters in these variables.



- Python variables do not need explicit declaration to reserve memory space.
- The declaration happens automatically when you assign a value to a variable.
- The equal sign (=) is used to assign values to variables.
- The operand to the left of the = operator is the name of the variable and the operand to the right of the = operator is the value stored in the variable.



```

20
21 '''
22 Assignment of variables
23
24 '''
25
26 x=10    # Now x become a variable with integer data type
27 y=100.0 # Now y become a variable with float data type
28 z='venkat' # Now z become a variable with string data type
29
30 print(x)
31 print(y)
32 print(z)

```

Multiple Assignments

- ✓ Python assigns single value to multiple variables
- ✓ Python assigns multiple values to multiple variables

```
In [1]: x=y=z=10  
print("x=",x)  
print("y=",y)  
print("z=",z)
```

```
x= 10  
y= 10  
z= 10
```

```
In [2]: x,y,z=10,100.0,'venkat'  
print("x=",x)  
print("y=",y)  
print("z=",z)
```

```
x= 10  
y= 100.0  
z= venkat
```



Standard Data Types

- ✓ Number
- ✓ List
- ✓ String
- ✓ Tuple
- ✓ Dictionary



- ✓ Number data types store numeric values.
- ✓ Number objects are created when you assign a value to them

```
In [4]: a=1  
        b=2  
        print("a=",a)  
        print("b=",b)
```

```
a= 1  
b= 2
```

- ✓ Can also delete the reference to a number object by using the **del** statement.

```
In [11]: a=1  
         b=2  
         print("a=",a)  
         print("b=",b)
```

```
a= 1  
b= 2
```

```
In [12]: del a  
         print("b=",b)  
         print("a=",a)
```

```
b= 2
```

```
-----  
NameError                                Traceback (most recent call last)  
<ipython-input-12-7b983382412d> in <module>()  
      1 del a  
      2 print("b=",b)  
----> 3 print("a=",a)
```

```
NameError: name 'a' is not defined
```

- ✓ Can delete a single object or multiple objects by using the **del** statement.

```
In [13]: a=1
         b=2
         print("a=",a)
         print("b=",b)
         del a,b # Delete both a and b objects
         print("a=",a)
         print("b=",b)
```

```
a= 1
b= 2
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-13-4434fb3bf339> in <module>()
      4 print("b=",b)
      5 del a,b # Delete both a and b objects
----> 6 print("a=",a)
      7 print("b=",b)
```

```
NameError: name 'a' is not defined
```



- ✓ Python supports three different numerical values
- Signed integer
 - float
 - Complex Number



```
In [17]: rvalue=2  
         ivalue=3  
         z=complex(rvalue,ivalue)  
         print("z=",z)
```

z= (2+3j)



Operations on Complex Number

```
In [23]: realpart=z.real
         imagpart=z.imag
         conjugatevalue=z.conjugate() # Conjugate of complex number
         magnitude=abs(z) # absolute value of a complex number
         print("magnitude=",magnitude)
         print("conjugatevalue=",conjugatevalue)

         z1=complex(2,3)
         z2=complex(5,6)
         z3=z1+z2 # Addition of complex numbers
         z4=z1-z2 # Substraction of complex numbers
         z5=z1*z2 # Multiplication of complex numbers
         z6=z1/z2 # Division of complex numbers

         print("Addition of complex numbers=",z3)
         print("Substraction of complex numbers=",z4)
         print("Multiplication of complex numbers=",z5)
         print("Division of complex numbers=",z6)

magnitude= 3.605551275463989
conjugatevalue= (2-3j)
Addition of complex numbers= (7+9j)
Substraction of complex numbers= (-3-3j)
Multiplication of complex numbers= (-8+27j)
Division of complex numbers= (0.4590163934426229+0.04918032786885245j)
```

- ✓ In-order to measure angle in complex number we need to import **numpy** package

```
In [29]: import numpy as np  
z1=complex(2,3)  
theta=np.angle(z1)  
print("Angle in Radians",theta)
```

Angle in Radians 0.982793723247329

- ✓ Strings in Python are identified as a contiguous set of characters represented in the quotation marks.
- ✓ Python allows either pair of single or double quotes
- ✓ Subsets of strings can be taken using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the string and working their way from -1 to the end.



```
In [50]: str= 'Hi I am Venkataramana'
print(str)      # It will print complete string
print(str[0])   # It will print only first charecter
print(str[0:2]) # It will print Hi without space
print(str[0:3]) # It will print Hi with space
print(str[3:])  # It will print from I
```

Hi I am Venkataramana

H

Hi

Hi

I am Venkataramana



```
In [55]: str= 'Hi I am Venkataramana'
print(str * 2)           # It will print complete string twice
print(str +'call me')    # It will append call me
print(str +' ' + 'call me') # It will append with space
print(str[::-1])         # It will print in reverse
print(str[20:0:-1])       # It will print in reverse upto i
```

```
Hi I am VenkataramanaHi I am Venkataramana
Hi I am Venkataramanacall me
Hi I am Venkataramana call me
anamaratakneV ma I iH
anamaratakneV ma I i
```



- ✓ Lists are the most versatile of Python's compound data types.
- ✓ A list contains items separated by commas and enclosed within square brackets ([]).like ['rama', 123, 100.0]
- ✓ To some extent, lists are similar to arrays in C. One of the differences between them is that all the items belonging to a list can be of different data type.
- ✓ The values stored in a list can be accessed using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the list and working their way to end -1.
- ✓ The plus (+) sign is the list concatenation operator
- ✓ the asterisk (*) is the repetition operator.



```
In [59]: Qualification=['venkat','B.Tech',2006,'M.E.',2011,'Ph.D',2018]
Experience=['SSCE',2007,'infosys','2011','AITAM','2013','SREC','2018']
CA=Qualification + Experience           # Concatination of the two lists
print("Qualification=",Qualification)
print("Experience=",Experience)
print("CA=",CA)

Qualification= ['venkat', 'B.Tech', 2006, 'M.E.', 2011, 'Ph.D', 2018]
Experience= ['SSCE', 2007, 'infosys', '2011', 'AITAM', '2013', 'SREC', '2018']
CA= ['venkat', 'B.Tech', 2006, 'M.E.', 2011, 'Ph.D', 2018, 'SSCE', 2007,
```



```
: Qualification=['venkat','B.Tech',2006,'M.E.',2011,'Ph.D',2018]
print(Qualification)           #Print Qualification
print(Qualification[0])        #Print first item in the list
print(Qualification[1:])       #Print all items in the list except first one
print(Qualification[1:5])      #Print second to fifth items from list
print(Qualification * 2)       #Print List twice
print(Qualification[::-1])     #Print List reverse
```

['venkat', 'B.Tech', 2006, 'M.E.', 2011, 'Ph.D', 2018]
venkat
['B.Tech', 2006, 'M.E.', 2011, 'Ph.D', 2018]
['B.Tech', 2006, 'M.E.', 2011]
['venkat', 'B.Tech', 2006, 'M.E.', 2011, 'Ph.D', 2018, 'venkat', 'B.Tech', 2006,
[2018, 'Ph.D', 2011, 'M.E.', 2006, 'B.Tech', 'venkat']



Update the list

```
list=['venkat','1985']  
print('Venkat birth year',list[1])  
list[0]='dev'  
list[1]=2017  
print('dev birth year',list[1])  
print(list)
```

Venkat birth year 1985
dev birth year 2017
['dev', 2017]

Delete list

```
list=['venkat','1985']  
print(list)  
del list[1]  
print(list)
```

```
['venkat', '1985']  
['venkat']
```



```
Name=[ 'Ramana', 'Thanmai', 'Dev' ]
DOB=[1985,2014,2017]
print('Number of peoples',len(Name))
print('=====')
Details=Name+DOB # Concatination
print('Biodata',Details)
print('=====')
Repetition=Name*2
print('Repetition',Repetition)
print('=====')
for x in Name:
    print(x)
print('=====')
```



Number of peoples 3

=====

Biodata ['Ramana', 'Thanmai', 'Dev', 1985, 2014, 2017]

=====

Repetition ['Ramana', 'Thanmai', 'Dev', 'Ramana', 'Thanmai', 'Dev']

=====

Ramana

Thanmai

Dev

=====

```
Name=[ 'Ramana', 'Thanmai', 'Dev' ]  
DOB=[1985,2014,2017]  
print('Name with maximum charecters',max(Name))  
print('Younger person DOB',max(DOB))  
print('Name with minimum charecters',min(Name))  
print('Older person DOB',min(DOB))
```

Name with maximum charecters Thanmai
Younger person DOB 2017
Name with minimum charecters Dev
Older person DOB 1985

This method converts tuple or string into a list

```
atuple=('ramana',1985,'Thanmai',2014)
str='venkat'
list1=list(atuple)
list2=list(str)
print(list1)
print(list2)
```

```
['ramana', 1985, 'Thanmai', 2014]
['v', 'e', 'n', 'k', 'a', 't']
```



List methods: list.append()

```
Students=['ram','venkat']  
print(Students)  
Students.append('Dev')  
print(Students)
```

```
['ram', 'venkat']  
['ram', 'venkat', 'Dev']
```

List methods: list.count()

```
Grade=['A','A','B','C','A']  
print('How many peoples got A grade',Grade.count('A'))
```

How many peoples got A grade 3



List methods: list.extend()

Method does not return any value but add the content to existing list

```
Family=['Venkat','Swetha']  
print('Family members',Family)  
atuple=('Dev','Thanmai')  
Family.extend(atuple)  
print('Family members',Family)
```

```
Family members ['Venkat', 'Swetha']  
Family members ['Venkat', 'Swetha', 'Dev', 'Thanmai']
```

Returns the lowest index in list that obj appears

```
Grade=['A','A','B','C','A','B']  
print('lowest index for grade B',Grade.index('B'))
```

lowest index for grade B 2

This method does not return any value but it inserts the given element at the given index.

```
Name= ['Ramana', 'Thanmai', 'Dev']  
print('Family members', Name)  
Name.insert(1, 'Swetha')  
print('Family members', Name)
```

Family members ['Ramana', 'Thanmai', 'Dev']

Family members ['Ramana', 'Swetha', 'Thanmai', 'Dev']



This method returns the removed object from the list.

```
Family=['Ramana', 'Swetha', 'Thanmai', 'Dev']  
print('Family members',Family)  
Family.pop()  
print('Family members',Family)  
Family.pop(2)  
print('Family members',Family)
```

```
Family members ['Ramana', 'Swetha', 'Thanmai', 'Dev']  
Family members ['Ramana', 'Swetha', 'Thanmai']  
Family members ['Ramana', 'Swetha']
```

list.remove()

This method
does not return any value but removes the given object from the list.

```
Family=['Ramana', 'Swetha', 'Thanmai', 'Dev']  
print('Family members',Family)  
Family.remove('Ramana')  
print('Family members',Family)
```

```
Family members ['Ramana', 'Swetha', 'Thanmai', 'Dev']  
Family members ['Swetha', 'Thanmai', 'Dev']
```



This method does not return any value but reverse the given object from the list.

```
Family=['Ramana', 'Swetha', 'Thanmai', 'Dev']  
print('Family members',Family)  
Family.reverse()  
print('Family members',Family)
```

```
Family members ['Ramana', 'Swetha', 'Thanmai', 'Dev']  
Family members ['Dev', 'Thanmai', 'Swetha', 'Ramana']
```

This method does not return any value but sort the given object in the list.

```
Family=['Ramana', 'Swetha', 'Thanmai', 'Dev']  
print('Family members',Family)  
Family.sort()  
print('Family members',Family)
```

```
Family members ['Ramana', 'Swetha', 'Thanmai', 'Dev']  
Family members ['Dev', 'Ramana', 'Swetha', 'Thanmai']
```

c:\n



- ✓ **Tuple** is similar to **List**, Difference is in **List** elements are enclosed in **square bracket** whereas in **Tuples** elements are enclosed in **parenthesis**
- ✓ In **Tuple**, elements cannot be updated unlike **List**. Data in **Tuple** is read only data



```
tuple=('venkat','B.Tech',2006)
print(tuple)           # Print Tuple
print(tuple[0])        # Print first element in the Tuple
print(tuple[0:2])      # Print first two elements
print(tuple[0:])       # Print All elements
print(tuple[::-1])     # Print all elements in reverse order
print(tuple[2:0:-1])   # Print elements in reverse order upto second elem
```

```
('venkat', 'B.Tech', 2006)
venkat
('venkat', 'B.Tech')
('venkat', 'B.Tech', 2006)
(2006, 'B.Tech', 'venkat')
(2006, 'B.Tech')
```



```
tuple=('venkat','B.Tech',2006) # First tuple
pgtuple=('M.E','2011') # Second Tuple
```

```
print(tuple*2) # Print tuple twice
print(tuple + pgtuple) # Tuple concatenation
```

```
List=['venkat','Ph.D','2018']
print(List)
```

```
List[0]='Ramana'
print(List)
tuple[0]='Ramana'
```

```
('venkat', 'B.Tech', 2006, 'venkat', 'B.Tech', 2006)
('venkat', 'B.Tech', 2006, 'M.E', '2011')
['venkat', 'Ph.D', '2018']
['Ramana', 'Ph.D', '2018']
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-14-196bd710262b> in <module>()
     10 List[0]='Ramana'
     11 print(List)
--> 12 tuple[0]='Ramana'
```

TypeError: 'tuple' object does not support item assignment

- Dictionaries are enclosed by **curly braces**(`{ }`) and values can be assigned and accessed using **square braces** (`[ ]`)



```
dict={} # Create empty dictionary
print(dict) # Print dictionary
dict['one']='This is one' # Update dictionary
print(dict) # Print Updated dictionary
dict['2']='This is two' # Update dictionary
print(dict) # Print Updated dictionary
print(dict.keys()) # Print keys like 'one','2'
print(dict.values()) # Print values like 'This is one'
```

```
{}
```

```
{'one': 'This is one'}
```

```
{'one': 'This is one', '2': 'This is two'}
```

```
dict_keys(['one', '2'])
```

```
dict_values(['This is one', 'This is two'])
```



```
tiny={'Name':'Venkat','Marks':891}# Create dictionary
print(tiny)# Print dictionary
print(tiny['Name'])# Print value in dictionary corre key 'Name'
print(tiny['Marks'])# Print value in dictionary corre key 'Marks'
tiny['Name']='ramana'# Update name in dictionary
print(tiny) # Print Updated dictionary
```

```
{'Name': 'Venkat', 'Marks': 891}
Venkat
891
{'Name': 'ramana', 'Marks': 891}
```



Convert String to int using int()

```
x=input('enter value of x') # by default x is a string
y=input('enter value of y') # By default y is a string
z=x+y # String Concatination
print(z) # Print Z
z1=int(x)+int(y) # Conversion from string to int
print(z1) # Print Z1
```

```
enter value of x10
enter value of y100
10100
110
```

Convert String to float using float()

```
x=input('enter value of x') # by default x is a string
y=input('enter value of y') # By default y is a string
z=x+y # String Concatination
print(z) # Print Z
z1=float(x)+float(y)# Conversion from string to int
print(z1) # Print Z1
print(float(x))
```

```
enter value of x10
enter value of y100
10100
110.0
10.0
```

Convert number to string using str()

```
x=200  
y=300  
z=x+y  
print(z)  
z1=str(x)+str(y)  
print(z1)
```

500

200300



Convert number to hexa decimal and octal string

```
x=200
print("x=",x)
y=hex(x)
print("hexa decimal string=",y)
y=oct(x)
print("octa decimal string=",y)
```

x= 200

hexa decimal string= 0xc8

octa decimal string= 0o310

Types of Operators in Python

- ✓ Arithmetic Operators
- ✓ Relational Operators
- ✓ Assignment Operators
- ✓ Logical Operators
- ✓ Bitwise Operators
- ✓ Membership Operators
- ✓ Identity Operators



```
x=20  
y=5  
add=x+y # Addition  
sub=x-y # Subtraction  
Mul=x*y # Multiplication  
Div=x/y # Division  
Rem=x%y # Remainder  
power=x**y # x power y
```



```
print("Addition=",add)
print("Substraction=",sub)
print("Mulitplication=",Mul)
print("Division=",Div)
print("Remainder=",Rem)
print("x power y=",power)
```

Addition= 25

Substraction= 15

Mulitplication= 100

Division= 4.0

Remainder= 0

x power y= 3200000



- The division of operands where the result is the quotient in which the digits after the decimal point are removed.
- if one of the operands is negative, the result is floored, i.e., rounded away from zero (towards negative infinity)

```
x=5  
y=-2  
z=x//y  
print("floor divison=",z)
```

floor divison= -3



Operator	Description
==	If the values of two operands are equal then the condition becomes true.
!=	If values of two operands are not equal, then condition becomes true.
>	If the value of left operand is greater than the value of right operand, then condition becomes true.
<	If the value of left operand is less than the value of right operand, then condition becomes true.
>=	If the value of left operand is greater than or equal to the value of right operand, then condition becomes true.
<=	If the value of left operand is less than or equal to the value of right operand, then condition becomes true.



```
a=20
b=10
c=a+b # Add a and b and result assigned to c
print("c=",c)
b+=a # b=b+a
print("b=",b)
c-=a # c=c-a
print("c=",c)
```

c= 30

b= 30

c= 10

```
x=100  
y=10  
y*=x #y=y*x  
print("y=",y)  
y/=x #y=y/x  
print("y=",y)  
y%=x #y=y%x  
print("y=",y)
```

y= 1000

y= 10.0

y= 10.0



SR
ENGINEERING COLLEGE



Convert Integer to binary using bin()

```
x=100
y=bin(x) # Convert integer into binary
print("y=",y)
z1=bin(x << 2) # Shift binary by two bits towards left
print("z1=",z1)
z2=bin(x >> 2) # Shift binary by two bits towards right
print("z2=",z2)
```

```
y= 0b1100100
z1= 0b110010000
z2= 0b11001
```

if operand a is true and b is false

Operator	Description	Example
and	If both the operands are true then condition becomes true.	(a and b) is False.
or	If any of the two operands are non-zero then condition becomes true.	(a or b) is True.
not	Used to reverse the logical state of its operand.	Not(a and b) is Tr

Python's membership operators test for membership in a sequence, such as strings, lists, or tuples.

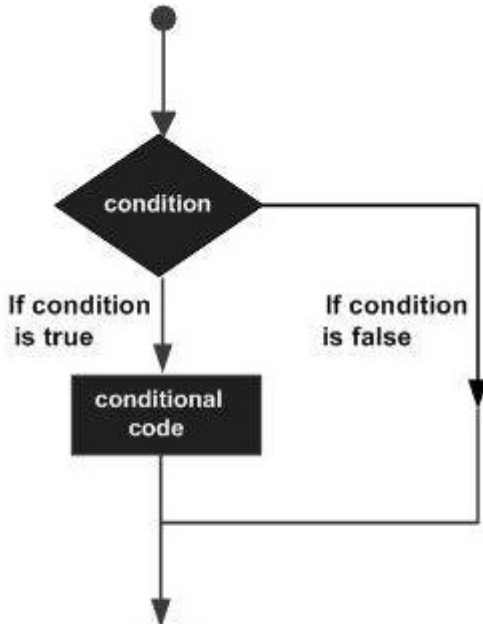
Operator	Description
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.
not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.



- Identity operators compare the memory locations of two objects

	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here is results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y).

Decision Making: if statement



Decision Making: if statement

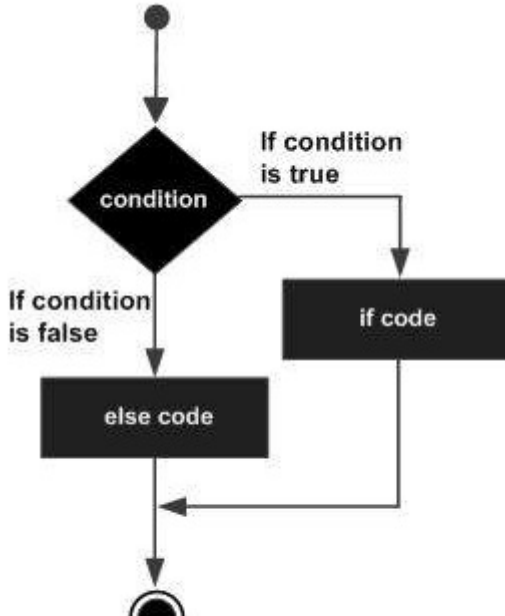
```
a=10
```

```
if(a==10):  
    print("a=",a)  
    print("Good bye")
```

a= 10

Good bye

Decision Making: if-else statement



Decision Making: if-else statement

```
a=10
if(a==20):
    print("a=",a)
    print("HI")
else:
    print("Good Bye")
```

Good Bye



Decision Making: nested if-else statement

```
x=input("enter value add=1 sub=2 mul=3 div=4")
a=20
b=30
if(x==1):
    print("add=",a+b)

else:
    if(x==2):
        print("sub=",a-b)
    else:
        if(x==3):
            print("mul=",a*b)
        else:
            print("div=",a/b)
```



```
enter value add=1 sub=2 mul=3 div=44
div= 0.6666666666666666
```

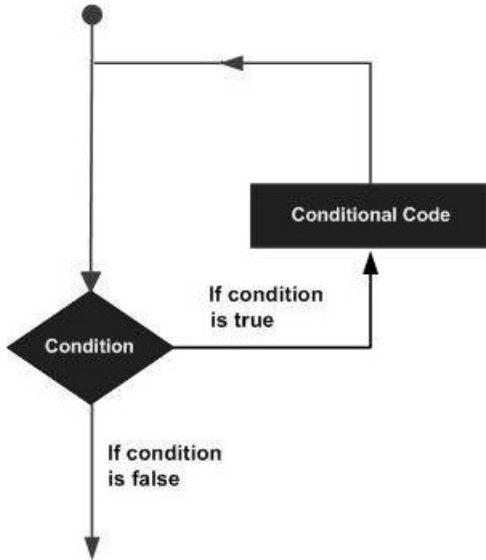


Decision Making: nested elif statement

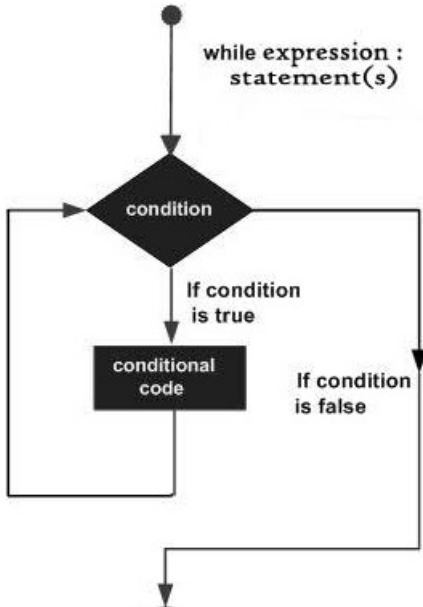
```
amount=1000
if(amount<500):
    discount=0.05
    amount=amount-discount*amount
elif (amount >= 500) and (amount < 1000):
    discount=0.1
    amount=amount-discount*amount
else:
    discount=0.2
    amount=amount-discount*amount
print("amount=",amount)
```

amount= 800.0

Loops



While Loop



While Loop

- ✓ Syntax
while (condition):
statement(s)

```
count=0  
while(count<=5):  
    print("count=",count)  
    count=count+1  
print("good bye")
```

```
count= 0  
count= 1  
count= 2  
count= 3  
count= 4  
count= 5  
good bye
```



While Loop with else statement

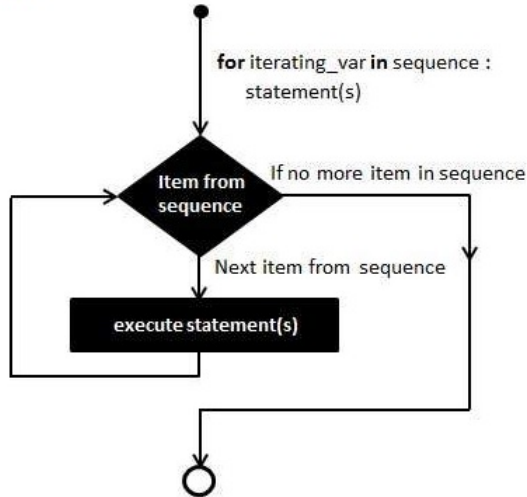
```
count=0
while(count<=5):
    print("count=",count)
    count=count+1
else:
    print("good bye")
```

```
count= 0
count= 1
count= 2
count= 3
count= 4
count= 5
good bye
```



For Loop flowchart

Flow Diagram



Email ID: venkataramana_v@srecwarangal.ac.in



✓ Syntax

- for variable in sequence
- statements

```
for i in range(5):  
    print("i=",i)
```

i= 0

i= 1

i= 2

i= 3

i= 4



For loop

```
for letter in 'ramana':  
    print("letter=",letter)  
print('good bye')
```

```
letter= r  
letter= a  
letter= m  
letter= a  
letter= n  
letter= a  
good bye
```



For loop

```
fruits=['Apple', 'Banana', 'Grapes']  
for i in fruits:  
    print("fruit=",i)  
print("good bye")
```

```
fruit= Apple  
fruit= Banana  
fruit= Grapes  
good bye
```

For loop

```
fruits=['Apple', 'Banana', 'Grapes']  
print("number of fruits=",len(fruits))  
for fruit in range(len(fruits)):  
    print('fruit name=',fruits[fruit])  
print("good bye")
```

```
number of fruits= 3  
fruit name= Apple  
fruit name= Banana  
fruit name= Grapes  
good bye
```



For loop

Separate even and odd numbers from 1 to 10

```
for i in range(11):  
    if(i%2 == 0):  
        print('even number=',i)  
    else:  
        print('odd number=',i)
```

```
even number= 0  
odd number= 1  
even number= 2  
odd number= 3  
even number= 4  
odd number= 5  
even number= 6  
odd number= 7  
even number= 8  
odd number= 9  
even number= 10
```



- SYNTAX -1

```
for i in range(10):  
    for j in range(20):  
        statement(s)  
    statement(s)
```

- SYNTAX -2

```
while (condition1):  
    while (condition2):  
        statement(s)  
    statement(s)
```



Nested loops

```
for i in range(1,11):  
    for j in range(1,11):  
        k=i*j  
        print(k, end=' ')  
    print()
```

```
1 2 3 4 5 6 7 8 9 10  
2 4 6 8 10 12 14 16 18 20  
3 6 9 12 15 18 21 24 27 30  
4 8 12 16 20 24 28 32 36 40  
5 10 15 20 25 30 35 40 45 50  
6 12 18 24 30 36 42 48 54 60  
7 14 21 28 35 42 49 56 63 70  
8 16 24 32 40 48 56 64 72 80  
9 18 27 36 45 54 63 72 81 90  
10 20 30 40 50 60 70 80 90 100
```



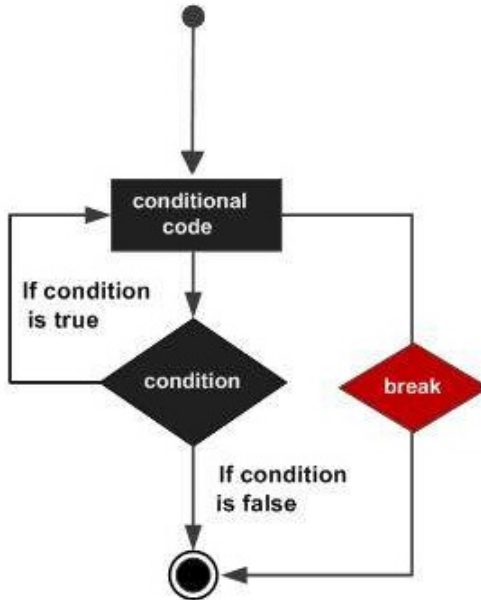
- ✓ break statement
 - Terminates the loop statement and transfers execution to the statement immediately following the loop.
- ✓ continue statement
 - Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.
- ✓ pass statement
 - The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.



- ✓ The break statement is used for premature termination of the current loop
- ✓ After abandoning the loop, execution at the next statement is resumed, just like the traditional break statement in C
- ✓ The break statement can be used in both while and for loops.
- ✓ If you are using nested loops, the break statement stops the execution of the innermost loop and starts executing the next line of the code after the block.



Break statement



Break statement

Identify index of number '5' in [10,2,6,5,12]

```
num=[10,2,6,5,12]
for i in range(len(num)):
    if(num[i]==5):
        print("index=",i)
print('good bye')
```

```
index= 3
good bye
```



Break statement

```
var=10
while(var>0):
    print("var=",var)
    if(var==5):
        break
    var=var-1
print('Good Bye')
```

```
var= 10
var= 9
var= 8
var= 7
var= 6
var= 5
Good Bye
```



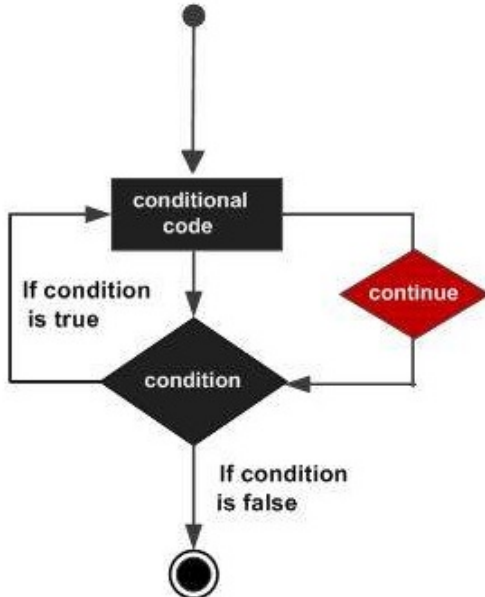
Break statement

```
no=input('enter numerical value')
num=[2,3,20,41,52,95,62]
for i in num:
    if(i==no):
        print("number found")
        break
else:
    print("number not found")
```

enter numerical value10
number not found

- ✓ The continue statement in Python returns the control to the beginning of the current loop.
- ✓ When encountered, the loop starts next iteration without executing the remaining statements in the current iteration.
- ✓ The continue statement can be used in both while and for loops.





continue statement

```
str='ramana'

for i in str:
    if(i=='m'):
        continue
    print("letter=",i)
print("good bye")
```

```
letter= r
letter= a
letter= a
letter= n
letter= a
```

Email ID: venkataramana_v@srecwarangal.ac.in



continue statement

```
var=10
while(var>0):
    var=var-1
    if(var>5):
        continue
    print("var=",var)
print("good bye")
```

```
var= 5
var= 4
var= 3
var= 2
var= 1
var= 0
good bye
```

Email ID: venkataramana_v@srecwarangal.ac.in



- ✓ It is used when a statement is required syntactically but you do not want any command or code to execute.
- ✓ The pass statement is a null operation; nothing happens when it executes.



Pass Statement

```
# Pass Block

language='Python'

for i in language:
    if(i=='h'):
        pass # Pass statement
        print('print this block')
    print("i=",i)
print('Good Bye')
```

```
i= P
i= y
i= t
print this block
i= h
i= o
i= n
Good Bye
```



- ✓ Iterator is an object which allows a programmer to traverse through all the elements of a collection, regardless of its specific implementation.
- ✓ Iterator object implements two methods like **iter()** and **next()**
- ✓ String, List or Tuple objects can be used to create Iterator



```
List=['ramana','SREC',2018]
it=iter(List) # Iterator Object Created
for i in it:|
    print('i=',i)
print('good bye')
```

```
i= ramana
i= SREC
i= 2018
good bye
```

```
List=['venkat','NITW',2014]
it=iter(List)
el=len(List)
while(el>0):
    print(next(it))
    el-=1
print('good bye')
```

venkat
NITW
2014
good bye



<code>abs(x)</code>	The absolute value of x
<code>ceil(x)</code>	The ceiling of x
<code>cmp(x,y)</code>	$-1:x < y, 0:x = y, 1:x > y$
<code>exp(x)</code>	Exponential of x
<code>fabs(x)</code>	The absolute value of x
<code>log(x)</code>	Natural log
<code>log10(x)</code>	10 base log
<code>max(x1,x2)</code>	Maximum value
<code>min(x1,x2)</code>	Minimum Value
<code>modf(x)</code>	The integer part is returned as a float
<code>pow(x,y)</code>	x power y
<code>round(x,[n])</code>	rounded upto n decimal
<code>sqrt(x)</code>	Square root of x



- ✓ The abs() method returns the absolute value of x i.e. the positive distance between x and zero.

```
x=-5;  
print("x value is =",x)  
print("absolute value of c is =",abs(x))
```

x value is = -5

absolute value of c is = 5

- ✓ The ceil() method returns the ceiling value of x i.e. the smallest integer not less than x.
- ✓ This function is not accessible directly, so we need to import **math** module and then we need to call this function using the math static object like **mt.ceil(x)**

```
import math as mt
x=2.5
y=mt.pi
z=-5.4
print("ceil of x = ",mt.ceil(x))
print("ceil of y = ",mt.ceil(y))
print("ceil of z = ",mt.ceil(z))
```

```
ceil of x = 3
ceil of y = 4
ceil of z = -5
```



Comparison of numbers

- Let X and Y are two numbers
- If $X > Y$ true then it returns 1
- If $X > Y$ false then it returns 0

```
x=5
y=4
...

Comparison of numbers
Here x>y true so it is 1
Here x<y false so it is 0
z=1-0
...

z=(x>y)-(x<y)
print('z=',z)
```

z= 1

Email ID: venkataramana_v@srecwarangal.ac.in



```
x=5
```

```
y=6
```

```
...
```

Comparison of numbers

Here $x > y$ true so it is 1

Here $x < y$ false so it is 0

```
z=1-0
```

```
...
```

```
z=(x>y)-(x<y)
```

```
print('z=',z)
```

```
z= -1
```

Comparison of numbers

```
x=6
```

```
y=6
```

```
...
```

Comparison of numbers

Here $x > y$ true so it is 1

Here $x < y$ false so it is 0

```
z=1-0
```

```
...
```

```
z=(x>y)-(x<y)
```

```
print('z=',z)
```

```
z=0
```



```
import math as mt  
x=20  
print('exponential of x = ',mt.exp(x))
```

exponential of x = 485165195.4097903



- ✓ fabs() similar to abs() which provides absolute value of input value
- ✓ abs() is built function whereas fabs() is defined in math module
- ✓ abs() working on integers, floats and complex numbers whereas fabs() working on integers and floats



```
import math as mt
print('absolute value is =',mt.fabs(-2.5))
print('absolute value is =',mt.fabs(-3))
print('abs of complex is =',abs(complex(3,4)))
print('abs of complex is =',mt.fabs(complex(3,4)))
```

absolute value is = 2.5

absolute value is = 3.0

abs of complex is = 5.0

TypeError

Traceback (most

<ipython-input-14-c71bafb7aa2a> in <module>()

3 print('absolute value is =',mt.fabs(-3))

4 print('abs of complex is =',abs(complex(3,4)))

----> 5 print('abs of complex is =',mt.fabs(complex(3,4)))

- ✓ The floor(x) method returns the floor of x i.e. the largest integer not greater than x.

```
import math as mt
x=2.5
y=-2.7
print('floor of x is = ',mt.floor(x))
print('floor of y is = ',mt.floor(y))
```

```
floor of x is = 2
floor of y is = -3
```

```
import math as mt
print('natural log is = ',mt.log(mt.e))
print('log base 10 is = ',mt.log10(10))
```

natural log is = 1.0

log base 10 is = 1.0

Min and Max Functions

```
list=[10, 20, 5, 62, 51, 32]  
print('max value = ',max(list))  
print('min value = ',min(list))
```

```
max value = 62  
min value = 5
```

- ✓ This method returns the fractional and integer parts of x in a two-item tuple. Both the parts have the same sign as x. The integer part is returned as a float.
- ✓ This one is part of math module

```
import math as mt
x=110.9
y=100
print('z1=',mt.modf(x))
print('z2=',mt.modf(y))
```

```
z1= (0.900000000000000057, 110.0)
z2= (0.0, 100.0)
```



- ✓ pow(x,y) is the command used to compute x power y
- ✓ It requires math module.

```
import math as mt
x=10
y=2
print('x power y is = ',mt.pow(x,y))
```

x power y is = 100.0

- ✓ round(x,n) round the floating value x upto n decimals from decimal point
- ✓ default value of n is zero. It is built in function in python

```
x=100.235  
y=-100.236  
z=2.56  
print('round upto 1 decimal=',round(x,1))  
print('round upto 2 decimal=',round(y,2))  
print('round upto 0 decimal=',round(z))
```

```
round upto 1 decimal= 100.2  
round upto 2 decimal= -100.24  
round upto 0 decimal= 3
```



- ✓ sqrt(x) computes square root of 'x' if $x > 0$
- ✓ It requires math module.

```
import math as mt  
x=100  
print('square root of x = ',mt.sqrt(x))
```

square root of x = 10.0

Random Functions: choice()

- ✓ Random numbers are used for games, simulations, testing, security and privacy applications
- ✓ choice() function is used to generate a random number from a list
- ✓ It requires **random** module

```
import random
list=['ram','venkat','ramana']
size=len(list)
print('choose winner:',list[random.choice(range(size))])
print('choose number from 1 to 10:',random.choice(range(1,10)))
```

```
choose winner: venkat
choose number from 1 to 10: 6
```

randrange()

- ✓ randrange() will produce a random number with in the range
- ✓ syntax: randrange(start,stop,step)
- ✓ start: starting point with in the range
- ✓ stop: end point with in the range, it will not included
- ✓ step: step value for range

```
import random
print('random number from 50 to 53',random.randrange(50,53))
print('odd numbers from 1 to 10',random.randrange(1,10,2))
print('even numbers from 0 to 11',random.randrange(0,11,2))
```

```
random number from 50 to 53 50
odd numbers from 1 to 10 9
even numbers from 0 to 11 10
```

- ✓ random() will produce a random floating number between 0.0 and 1.0

```
import random  
print('random number=', random.random())
```

random number= 0.34348879682293176



- ✓ This function shuffles elements in list or tuple randomly
- ✓ It requires random module

```
import random
list=['venkat','ramana',2019]
tuple=('ram','venk',2018)
random.shuffle(list)
print('shuffled list=',list)
```

```
shuffled list= ['venkat', 'ramana', 2019]
```



- ✓ The uniform(x,y) method returns a random float r, such that x is less than or equal to r and r is less than y.
- ✓ It requires random module

```
import random  
print('random value between 10 and 20=',random.uniform(10,20))
```

random value between 10 and 20= 15.700192866507287

ic.in



<code>acos(x)</code>	Returns cos inverse in radians
<code>asin(x)</code>	Returns sin inverse in radians
<code>atan(x)</code>	Returns tan inverse in radians
<code>cos(x)</code>	Returns cos of x, x in radians
<code>sin(x)</code>	Returns sin of x, x in radians
<code>tan(x)</code>	Returns tan of x, x in radians
<code>atan(y,x)</code>	Returns tan inverse of y/x in radians
<code>hypot(x,y)</code>	Returns square root of x square and y square
<code>degrees(x)</code>	Converts x in radians into degrees
<code>radians(x)</code>	Converts x in degrees into radians

```
import math as mt
print('cos(x)=',mt.cos(math.pi/2))
print('sin(x)=',mt.sin(math.pi/2))
print('tan(x)=',mt.tan(math.pi/2))
print('asin(x)=',mt.asin(1))
print('acos(x)=',mt.acos(-1))
print('atan(x)=',mt.atan(1))
print('euclidian distance=',mt.hypot(3,4))
print('Tan inverse y by x =',mt.atan2(2,2))
print('Covert radians in to degrees',mt.degrees(math.pi))
print('Covert degrees in to radians',mt.radians(180))
```

cos(x)= 6.123233995736766e-17

sin(x)= 1.0

tan(x)= 1.633123935319537e+16

asin(x)= 1.5707963267948966

acos(x)= 3.141592653589793

atan(x)= 0.7853981633974483

euclidian distance= 5.0

Tan inverse y by x = 0.7853981633974483

Covert radians in to degrees 180.0

Covert degrees in to radians 3.141592653589793



```
import math as mt  
print('pi value=',mt.pi)  
print('exp value=',mt.exp(1))
```

pi value= 3.141592653589793
exp value= 2.718281828459045

- ✓ Strings are most popular type in python
- ✓ Creating string is easy, just write characters with single quotes(' ') or double quotes(" ")

```
var1='my name is venkat'  
var2='working as assistant professor'  
print('what is your name:',var1)  
print('what are you doing:',var2)
```

```
what is your name: my name is venkat  
what are you doing: working as assistant profes
```



Accessing the characters from string

```
var1='my name is venkat'  
var2='working as assistant professor'  
print('Name:',var1[11:17])  
print('Job:',var2[11:31])
```

Name: venkat

Job: assistant professor

Update string

```
var1='my name is venkat'  
var2='working as assistant professor'  
print('Name: ',var1[:11]+'ramana')  
print('Job:', 'He is'+ var2[10:31])
```

Name: my name is ramana

Job: He is assistant professor

Escape Characters



```
var1='my name is venkat'  
var2='working as assistant professor'  
print('What is your name and designation: ',var1[11:17]+var2[10:31])  
print('Print your name twice: ',var1[11:17]*2)  
  
if('n' in var1):  
    print('n exist')  
  
if('z' not in var2):  
    print('z not exist')
```

```
What is your name and designation:  venkat assistant professor  
Print your name twice:  venkatvenkat  
n exist  
z not exist
```



String Formatting Operator %

Format Symbol	Description
%c	Character
%s	String
%d	Signed decimal Integer
%u	Unsigned decimal Integer
%o	Octal Integer
%x	Hexadecimal Integer
%X	Hexadecimal Integer Uppercase
%e	Exponential
%E	Exponential in Uppercase
%f	Floating point real number
%g	The shorter of %f and %e
%G	The shorter of %f and %E



```
print('My name is %s and my ph.d grade is %c'%( 'venkat','A'))
print('My B.tech Marks:%0.2f, completed in %d'%(71.84,2006))
```

My name is venkat and my ph.d grade is A
My B.tech Marks:71.84, completed in 2006



Symbol	Functionality
*	argument specifies width or precision
-	Left justification
+	Display the sign
<sp>	leave the blank space before +number
#	add the octal leading '0' or hexa decimal leading '0x' or '0X'
0	Pad from left with zeros
%	%% leaves a single literal '%'
(var)	Mapping variable
m.n.	m is minimum total width n is minimum digits to display



- ✓ Python's triple quotes comes to the rescue by allowing strings to span multiple lines, including verbatim NEWLINES, TABs, and any other special characters.
- ✓ The syntax for triple quotes consists of three consecutive single or double quotes.



```
para_str = """this is a long string that is made up of
several lines and non-printable characters such as
TAB ( \t ) and they will show up that way when displayed.
NEWLINES within the string, whether explicitly given like
this within the brackets [ \n ], or just a NEWLINE within
the variable assignment will also show up.
"""
print (para_str)
```

this is a long string that is made up of
several lines and non-printable characters such as
TAB () and they will show up that way when displayed.
NEWLINES within the string, whether explicitly given like
this within the brackets [
], or just a NEWLINE within
the variable assignment will also show up.



```
print ('C:\\\\nowhere')  
print (r'C:\\\\nowhere')
```

C:\\nowhere

C:\\\\nowhere

Built-in String Methods: capitalize()

It will make first letter of the string as capital letter

```
str='venkat'  
print('my name:',str.capitalize())
```

my name: Venkat

Built-in String Methods: center(width,char)

The method center() returns centered in a string of length width. Padding is done using the specified fillchar. Default filler is a space.

```
str='venkat'  
print('my name:',str.center(10,'a'))  
print('my name:',str.center(10))
```

my name: aavenkataa

my name: venkat

Built-in String Methods: count(sub,start,end)

The method count(sub,start,end), counts no of times 'sub' string will come in a string.

```
var1='my name is venkat'  
print('How many time a come=',var1.count('a',0,len(var1)))
```

How many time a come= 2

Built-in String Methods: endswith(suffix,start,end)

The method endswith(suffix,start,end), returns true if the given string end with suffix else returns false

```
var1='my name is venkat'  
suffix='venkat'  
print('return: ',var1.endswith(suffix))  
print('return: ',var1.endswith(suffix,10))  
print('return: ',var1.endswith(suffix,0,len(var1)))  
print('return: ',var1.endswith(suffix,0,10))
```

```
return: True  
return: True  
return: True  
return: False
```

Built-in String Methods: expandtabs(tabsize)

The method `expandtabs(tabsize)`, is used to keep how many character spacing needs to put if use tab in string. Default no of characters is 8.

```
var1='my name is\t venkat'  
print('original string:'+var1)  
print('original string:'+var1.expandtabs(8))  
print('original string:'+var1.expandtabs(10))
```

```
original string:my name is      venkat  
original string:my name is      venkat  
original string:my name is          venkat
```

Built-in String Methods: find(sub,start,end)

The method find(sub,start,end), returns index if sub exist in given string else it will return -1

```
var1='my name is venkat'  
sub='venkat'  
print('return1:',var1.find(sub))  
print('return2:',var1.find(sub,0,len(var1)))  
print('return3:',var1.find(sub,0,10))
```

```
return1: 11  
return2: 11  
return3: -1
```

Built-in String Methods: index(sub,start,end)

The method find(sub,start,end), returns index if sub exist in given string else it will return exception

```
var1='my name is venkat'  
sub='venkat'  
print('return1:',var1.index(sub))  
print('return2:',var1.index(sub,0,len(var1)))  
print('return3:',var1.index(sub,0,10))
```

return1: 11

return2: 11

ValueError

Traceback (most recent call last):

```
<ipython-input-84-22e3bcf916fe> in <module>()  
      3 print('return1:',var1.index(sub))  
      4 print('return2:',var1.index(sub,0,len(var1)))  
----> 5 print('return3:',var1.index(sub,0,10))
```

Built-in String Methods: isalnum()

The method isalnum(), returns true if string contains atleast one character and string in alphanumeric

```
str1='my name is 007'  
print('return1:',str1.isalnum())  
str2='mynameis007'  
print('return2:',str2.isalnum())
```

return1: False

return2: True



Built-in String Methods: isalpha()

The method `isalpha()`, returns true if string contains atleast one character and string in complete alphabetic

```
str1='my name is venkat'  
print('return1:',str1.isalpha())  
str2='mynameis007'  
print('return2:',str2.isalpha())  
str3='mynameisvenkat'  
print('return3:',str3.isalpha())
```

```
return1: False  
return2: False  
return3: True
```



Built-in String Methods: isdigit()

The method isdigit() returns true if string contains only digits

```
str1='my name is venkat'  
str2='023410259'  
print('return1:',str1.isdigit())  
print('return2:',str2.isdigit())
```

return1: False

return2: True

Built-in String Methods: islower()

The method islower() returns true if string contains atleast one cased character and all characters are lower case

```
str1='my name is venkat'  
str2='my Name is venkat'  
str3='mynameisvenkat'  
str4='mynameisvenkat007'  
print('contains space:',str1.islower())  
print('contains upper case:',str2.islower())  
print('No space:',str3.islower())  
print('contains numeric:',str4.islower())
```

```
contains space: True  
contains upper case: False  
No space: True  
contains numeric: True
```

Email ID: venkataramana_v@srecwarangal.ac.in



Built-in String Methods: isupper()

The method `isupper()` returns true if string contains atleast one cased character and all characters are upper case

```
str1='my name is venkat'  
str2='my Name is venkat'  
str3='MY NAME IS VENKAT'  
str3='MY NAME IS VENKAT 259'  
print('return1:',str1.isupper())  
print('return2:',str2.isupper())  
print('return3:',str3.isupper())  
print('return4:',str4.isupper())
```

```
return1: False  
return2: False  
return3: True  
return4: False
```

Email ID: venkataramana_v@srecwarangal.ac.in



Built-in String Methods: isnumeric()

The method `isnumeric()` returns true if string contains only numerical

```
str1='my name is venkat'  
str2='023410259'  
str3='0234 10259'  
print('return1:',str1.isnumeric())  
print('return2:',str2.isnumeric())  
print('return3:',str3.isnumeric())
```

```
return1: False  
return2: True  
return3: False
```

Built-in String Methods: isspace()

The method `isspace()` returns true if string contains only white space characters

```
str1='my name is venkat'  
str2='023410259'  
str3='my name is venkat259'  
str4=' '  
print('return1:',str1.isspace())  
print('return2:',str2.isspace())  
print('return3:',str3.isspace())  
print('return4:',str4.isspace())
```

```
return1: False  
return2: False  
return3: False  
return4: True
```

Email ID: venkataramana_v@srecwarangal.ac.in



Built-in String Methods: join(seq)

The method join() returns a string with elements in sequence seq, each element separated by mentioned string

```
seq=['venkat','ramana','srec','2007']  
str='-'  
print('string1=',str.join(seq))  
str='AAA'  
print('string2=',str.join(seq))
```

string1= venkat-ramana-srec-2007

string2= venkatAAAramanaAAAsrecAAA2007

Built-in String Methods: ljust(width,fillchar)

The method ljust() returns a left justified string with width as length, if width is greater than length of original string then remaining elements filled with mentioned fillchar, original string was returned if width is less than length.

```
str1='venkat'  
print('return1:',str1.ljust(10,'*'))  
print('return2:',str1.ljust(5,'*'))
```

```
return1: venkat****  
return2: venkat
```



Built-in String Methods: rjust(width,fillchar)

The method `rjust()` returns a right justified string with width as length, if width is greater than length of original string then remaining elements filled with mentioned fillchar, original string was returned if width is less than length.

```
str1='venkat'  
print('return1:',str1.rjust(10,'*'))  
print('return2:',str1.rjust(5,'*'))
```

```
return1: ***venkat  
return2: venkat
```



Built-in String Methods: upper() and lower()

The method upper() converts complete string into upper case
The method lower() converts complete string into lower case

```
str1='venkat'  
print('return1:',str1.upper())  
str2='VENKAT'  
print('return2:',str1.lower())
```

return1: VENKAT

return2: venkat

Built-in String Methods: lstrip() and rstrip()

The method lstrip() trims left side of the string consists similar characters, default character is white space

The method rstrip() trims right side of the string consists similar characters, default character is white space

```
str2='   venkat   '  
print('return1:',str2.lstrip())  
print('return2:',str2.rstrip())  
str1='***venkat***'  
print('return3:',str1.lstrip('*'))  
print('return4:',str1.rstrip('*'))
```

```
return1: venkat  
return2:   venkat  
return3: venkat***  
return4: ***venkat
```

Email ID: venkataramana_v@srecwarangal.ac.in



Built-in String Methods: maketrans(str1,str2)

The method maketrans(), returns a translation table which maps each character in 'str1' with each character in same position in 'str2'.

Later table passed to translate method to convert a string.

```
str1='aciou'  
str2='12345'  
tab=str.maketrans(str1,str2)  
str='my name is venkataramana'  
print(str.translate(tab))
```

my n1me 3s venk1t1r1m1n1



max(str) returns maximum alphabetic in a string
min(str) returns minimum alphabetic in a string

```
str='venkataramana'  
print('maximum alphabetic in str',max(str))  
print('minimum alphabetic in str',min(str))
```

```
maximum alphabetic in str v  
minimum alphabetic in str a
```

This method replaces the old substring with new substring in a given string and returns new string

```
str='venkat born in a village, venkat did ssc in aprschool'
str1='venkat'
str2='ramana'
print(str.replace(str1,str2))
print(str.replace(str1,str2,1))
```

ramana born in a village, ramana did ssc in aprschool
ramana born in a village, venkat did ssc in aprschool

This method returns last index where given substring exist in a string

Whereas find() method returns first index where substring exist

```
str='my name is venkat, number is 59'
str1='is'
print('value1:',str.rfind(str1))
print('value2:',str.rfind(str1,0,len(str)))
print('value3:',str.find(str1))
print('value4:',str.find(str1,0,len(str)))
str2='ramana'
print('value1:',str.rfind(str2))
```

value1: 26

value2: 26

value3: 8

value4: 8

value1: -1

rindex()

This method is similar to rfind(), but it will through exception in case substring not found in string

```
str='my name is venkat, number is 59'  
str1='is'  
print('value1:',str.rindex(str1))  
print('value2:',str.rindex(str1,0,len(str)))  
print('value3:',str.index(str1))  
print('value4:',str.index(str1,0,len(str)))  
str2='ramana'  
print('value1:',str.rindex(str2))
```

```
value1: 26  
value2: 26  
value3: 8  
value4: 8
```

ValueError

Traceback (most recent call last)

```
<ipython-input-22-b840fc7aa5aa> in <module>()  
6 print('value4:',str.index(str1,0,len(str)))  
7 str2='ramana'  
----> 8 print('value1:',str.rindex(str2))
```



(rjust()) The rjust() method returns the string right justified in a string of length width. Padding is done using the specified fillchar (default is a space). The original string is returned if width is less than len()





Thank you!